

1. Purpose

This document specifies the screen-level UI/UX design for the SentinelCut Next.js frontend (`frontend/src/app/`), per Folder Structure §3). It defines layout, component composition, state, and interaction behavior for every route group (`(marketing)`, `(auth)`, `(app)`, `(admin)`), and — critically — how the confidence scores and detection sources produced by the AI Pipeline (Document 9, §11) are surfaced to a non-technical user in a way that builds trust rather than confusion. Every screen below is wired to a specific endpoint from the API Design document (Document 8); no screen in this document invents data the backend doesn't already provide.

This is a design *specification*, not a visual mockup file — it defines what exists on each screen, how it behaves, and why, so implementation can proceed directly against it using Tailwind CSS and the component libraries already scaffolded in `frontend/src/components/`.

2. Design Principles

1. **Confidence is not binary.** The AI Pipeline (Document 9 §10) deliberately persists every detection with a graded confidence score rather than a hard yes/no. The UI must never collapse that back into a flat "profanity found" list — the whole value of the Timeline Editor's human-in-the-loop design (Product Vision, PRD §4.7) depends on the user being able to see which detections are certain and which are borderline.
2. **The review step is the safety net, not an afterthought.** Given the accepted MVP accuracy bar ($\geq 85\%$ recall, per PRD §8/AI Pipeline §18), the UI's job is to make correcting the AI fast and low-friction, not to pretend the AI is perfect.
3. **Guest Mode must feel complete, not crippled.** A guest user should be able to run the full upload→detect→edit→export loop; the *only* difference in the UI is persistent history and presets (per PRD §4.1/FR-1.4), and this difference must be communicated, not hidden.
4. **One interaction model for light and dark.** Every screen is designed once, against design tokens (Section 3), not designed twice — dark mode is a token swap, not a parallel UI.
5. **Startup-SaaS visual bar, not academic-project visual bar.** Per the Master Prompt's explicit framing, density, motion, and typography choices below intentionally mirror what a funded SaaS product would ship, not a bare-bones CRUD scaffold.

3. Design Tokens & Visual System

Implemented as Tailwind CSS custom theme values in `frontend/tailwind.config.js`, consumed via CSS variables so light/dark switching (SRS §4.1, "Should priority") is a single class toggle on `<html>`.

3.1 Color Tokens

Token	Light	Dark	Usage
<code>--bg-base</code>	#FAFAFA	#0B0D10	App background
<code>--bg-surface</code>	#FFFFFF	#14171C	Cards, panels
<code>--bg-surface-raised</code>	#FFFFFF (shadow)	#1B1F26	Modals, dropdowns
<code>--text-primary</code>	#111318	#F1F2F4	Body text
<code>--text-secondary</code>	#5B6270	#9AA1AC	Meta text, labels
<code>--border-subtle</code>	#E6E8EB	#262B33	Dividers, card borders
<code>--brand-primary</code>	#6D5EF6 (indigo-violet)	#8B7CFF	Primary actions, active states
<code>--accent-teal</code>	#14B8A6	#2DD4BF	Success / completed states
<code>--accent-amber</code>	#F59E0B	#FBBF24	Borderline confidence, warnings
<code>--accent-rose</code>	#F43F5E	#FB7185	Destructive actions, high-severity detections

3.2 Confidence Color Scale (used exclusively in the Timeline Editor, Section 8)

This scale is the single most important visual mapping in the product, since it is the direct UI representation of AI Pipeline §10's `confidence_score`:

Confidence range	Color token	Meaning shown to user
≥ 0.85	<code>--accent-rose</code> (solid fill)	"High confidence"
0.60–0.84	<code>--accent-amber</code> (solid fill)	"Medium confidence — review suggested"
0.30–0.59	<code>--accent-amber</code> (outline only, no fill)	"Low confidence — likely needs review"
< 0.30 (only ever reachable via <code>min_confidence</code> filter override)	<code>--text-secondary</code> (dotted outline)	"Very low confidence"

3.3 Typography

- Primary typeface: **Inter** (variable weight) — chosen for its clarity in dense data UI (Dashboard, Timeline Editor transcript) and its native support for Latin + reasonable Devanagari fallback rendering.
- Devanagari text (Hindi transcript segments) renders with a secondary stack: `'Noto Sans Devanagari', Inter, sans-serif` to guarantee correct glyph rendering regardless of the user's OS font availability — critical given Hinglish/mixed-language transcripts render both scripts inline in the same sentence (AI Pipeline §6).
- Type scale: `text-xs` (12px, meta/timestamps) → `text-sm` (14px, transcript body) → `text-base` (16px, UI body) → `text-2xl`/`text-4xl` (headings, landing page).

3.4 Spacing & Radius

- 4px base spacing unit (Tailwind default scale, unmodified).
- Radius: `rounded-lg` (8px) for cards and inputs, `rounded-full` for pills/badges (confidence badges, status chips), `rounded-2xl` for modal containers — deliberately softer than a typical enterprise admin tool, reinforcing the "startup SaaS" tone from the Master Prompt rather than a bare academic form-grid look.

3.5 Dark Mode Implementation

`next-themes`-style class-based toggling (`class="dark"` on `<html>`), persisted client-side (in-memory/React state only inside artifacts context, but in the real app via a standard cookie/localStorage preference — no backend persistence needed for this preference). All components consume only the CSS variable tokens above; no component ever hardcodes a hex value, which is what makes the "designed once" principle in Section 2 actually enforceable in code review.

4. Global Layout & Navigation

4.1 Shell Structure

- **(marketing) layout:** Full-width, no persistent sidebar. Sticky top nav (logo, "Features," "Pricing," "Login," primary CTA "Try Free").
- **(auth) layout:** Centered single-column card on a subtly branded background (soft gradient using `--brand-primary` at low opacity), no nav — minimizes distraction during signup/login per SRS-NFR-USE-01's "no external instructions needed" goal.
- **(app) layout:** Persistent left sidebar (`components/shared/Sidebar.jsx`) + top bar (`components/shared/Navbar.jsx`) — search, notifications bell tied to job status, avatar/profile menu, dark-mode toggle). Sidebar items: Dashboard, Upload, History, Settings, and (conditionally) Admin, if `role === 'admin'` from the decoded JWT.
- **(admin) layout:** Same shell shape as `(app)` but with a distinct sidebar icon set and a persistent "Admin Mode" badge in the top bar, so an admin never confuses which context they're in while managing the profanity dictionary or user records.

4.2 Responsive Behavior (SRS §4.1: desktop + tablet minimum)

- **Desktop ($\geq 1024\text{px}$):** Sidebar always expanded, Timeline Editor uses a two-pane layout (transcript left, playback/waveform + detection detail panel right).
- **Tablet (768–1023px):** Sidebar collapses to icon-only rail (click to expand as overlay); Timeline Editor stacks vertically (playback bar pinned above a full-width scrolling transcript).
- **Mobile ($< 768\text{px}$):** Explicitly out of scope for full feature parity per SRS §4.1's minimum bar, but the Landing Page, Auth pages, and Dashboard degrade gracefully to single-column stacks so the marketing funnel and basic account checks aren't broken on a phone.

5. Landing Page (`(marketing)/page.js`)

Goal: Convert a visitor into either a Guest trial or a signup, per the Secondary Journey in PRD §6.2.

Sections (top to bottom):

1. **Hero** — headline framed around the Product Vision's core value prop ("Multilingual profanity censorship for English, Hindi & Hinglish content"), subheadline, two CTAs side by side: primary "Try as Guest" ($\rightarrow$ calls `POST /auth/guest-session`), then redirects straight to Upload — zero friction, matching PRD §6.2 step 1) and secondary "Sign Up Free."
2. **Live demo strip** — a short, pre-recorded before/after audio waveform showing a censored word (static asset, not a live pipeline call) to make the "beep replaces

profanity" concept immediately legible without requiring the visitor to upload anything.

3. **Language support band** — three icon-labeled pills: "English," "Hindi," "Hinglish / Mixed" — directly visualizing the Main Objective from the Master Prompt/Product Vision.
 4. **How it works** — 4-step horizontal stepper mirroring the User Flow diagram (Upload → Detect → Review → Export), each step illustrated with a small static screenshot of the corresponding real screen (Upload page, Timeline Editor, Export panel) rather than generic stock art — reinforces product credibility.
 5. **Detection modes explainer** — three cards (Automatic / Custom / Combined), directly describing PRD §4.4 in plain language.
 6. **Footer** — links to Privacy Policy, Terms & Conditions (per SRS-NFR-LEGAL-01), GitHub/roadmap links, and a "Built as an academic project" credibility note (optional, configurable — a startup-facing deployment could omit this).
-

6. Auth Screens (`(auth)/*`)

6.1 Signup (`(signup/page.js)`)

Fields: email, password (inline strength meter reflecting FR-1.5's 8-char/1-digit/1-special rule live as the user types — never a rejection *after* submit), display name (optional). Secondary action: "Continue with Google" button (calls `POST /auth/google`) placed *above* the email/password form, since OAuth is typically the lower-friction path — email signup is the fallback, not vice versa.

6.2 Login (`(login/page.js)`)

Email/password + Google button, "Forgot password?" link, and a persistent small-text link "Just want to try it? Continue as Guest" — the guest path is never buried, consistent with the zero-signup-friction persona (Ankit/Evaluator) called out in PRD §3.

6.3 Forgot / Reset Password

Two-step flow matching API Design §3.4/3.5 exactly: email-entry form → generic "if an account exists..." confirmation screen (never reveals whether the email exists, per the API's anti-enumeration design) → (via emailed link) a reset-token form with the same live strength meter as signup.

6.4 Guest Session Banner

Any screen reached via Guest Mode renders a persistent, dismissible-but-reappearing top banner: "You're in Guest Mode — your files expire in [live countdown from `(expires_at)`]. [Sign up to save your work]." This directly implements the PRD Edge Case requirement ("clear warning shown before expiry") and is a shared component (`(components/shared/GuestBanner.jsx)`) mounted once in the `(app)` layout rather than duplicated per page.

7. Dashboard ((app)/dashboard/page.js)

Consumes (GET /dashboard/summary) (API Design §8.1) in a single call on mount, rendering four zones:

1. **Stat strip** (top, 3-4 cards in a row): Credits Remaining, Storage Used (progress bar: (storage_used_bytes) / (storage_limit_bytes)), Active Jobs count. Each stat card is intentionally lightweight — no charts here, just numbers — charts live only in the Admin Analytics screen (Section 12).
2. **Processing Queue panel** — live list of (active_jobs), each row showing filename, current (stage) (from the WebSocket (status_update) payload, API Design §12.2), and a progress affordance. Rows subscribe individually to (WSS /ws/jobs/{job_id}) (falling back to 5s polling per API Design §12.4) and animate from (PROCESSING) → (READY_FOR_REVIEW) with a toast + row highlight, then auto-navigate the user into the Timeline Editor if they click the row.
3. **Recent Files grid** — card-per-file (thumbnail placeholder for video, waveform placeholder for audio-only), status pill, "Open" action routing to (/files/{id}/editor) if (READY_FOR_REVIEW)+, or (/files/{id}/processing) if still in progress.
4. **Empty state** (first-time users) — replaces zones 2-3 with a single large "Upload your first file" CTA card, avoiding a dashboard that looks broken/empty on day one.

8. Upload Flow

8.1 Upload Page ((app)/upload/page.js)

- **Dropzone** (components/upload/) — large drag-and-drop target plus "Browse files" fallback button; accepted-format list (MP4/MOV/MKV/AVI/MP3/WAV/AAC/FLAC/M4A) shown as small format chips *before* a file is dropped, not only as an error after rejection.
- **Client-side validation** runs immediately on file selection (extension + size against the configured max, mirroring SRS-FR-UPLD-04) — rejected files show a specific inline error ("This file is 620MB; the maximum is 500MB") per SRS-NFR-USE-03's "specific and actionable" requirement, never a generic "upload failed."
- **Upload progress:** determinate progress bar bound to the (multipart/form-data) upload's progress event, with a Cancel button that aborts the in-flight request (FR-3.5).
- On success ((201) from (POST /files/upload)), the page transitions in place (no full navigation) into:

8.2 Configuration Panel (same page, revealed after upload)

Two side-by-side selector groups, both required before "Start Processing" activates:

- **Detection Mode** (radio card group): Automatic / Custom / Combined. Selecting Custom or Combined reveals a textarea for comma/newline-separated words plus a "Load from preset ▾" dropdown (populated via `GET /presets`) — directly implementing PRD §4.4 and FR-DTCT-CFG-04.
- **Output Mode** (radio card group): Beep / Mute / Custom Sound. Selecting Custom Sound reveals a small secondary upload dropzone for the replacement clip.

"Start Processing" calls `POST /files/{id}/process` with the assembled `detection_mode`/`custom_words`/`preset_id`/`replacement_mode` body (API Design §4.5), then routes to the Processing Status page.

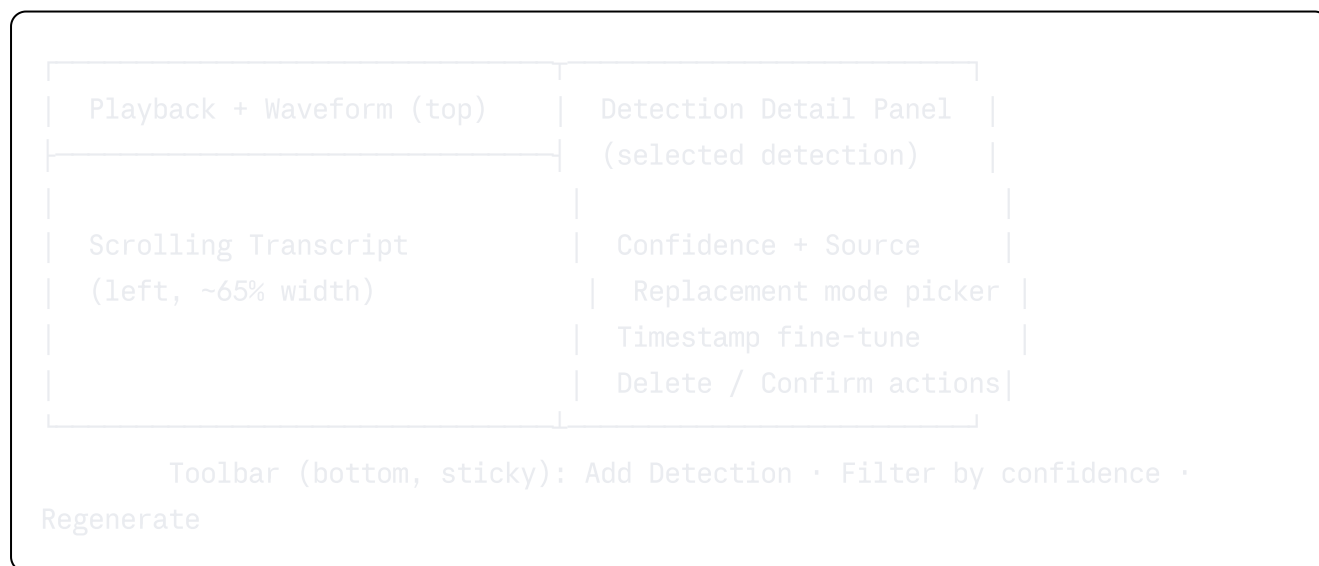
8.3 Processing Status Page (`files/[fileId]/processing/page.js`)

A focused, single-purpose screen (not the full dashboard) showing: file name, an animated stage indicator that maps directly onto the AI Pipeline stages (Document 9 §3) — "Extracting Audio → Transcribing Speech → Detecting Language → Scanning for Profanity" — driven by the `stage` field in each WebSocket `status_update` message, so the user watching this screen sees real pipeline progress, not a fake spinner. On `job_completed`, auto-navigates into the Timeline Editor.

9. Timeline Editor (`files/[fileId]/editor/page.js`)

This is the most technically demanding screen in the product (per System Architecture §3.1/Folder Structure §3 rationale) and the primary place where AI Pipeline §11's detection data must be made legible and editable. Layout described for desktop; tablet/mobile stacking per Section 4.2.

9.1 Layout



9.2 Transcript Rendering & Detection Highlighting (implements FR-TLE-01/02, consumes `GET /files/{id}/transcript` + `GET /files/{id}/detections`)

- Full transcript text renders as inline spans (`components/timeline-editor/TranscriptView.jsx`), each word positioned from `segments[].start/end` (API Design §6.1).
- Any span whose time range overlaps a `detections[]` entry (API Design §5.1) is wrapped in a `DetectionHighlight.jsx` component, background-filled or outlined per the **Confidence Color Scale** defined in Section 3.2 — this is the direct visualization of AI Pipeline §10's aggregated score, and is the single most load-bearing design decision in this document: a user must be able to glance at a paragraph and immediately see "one high-confidence hit, one borderline one" without opening anything.
- A small superscript badge on each highlighted span shows `detection_source` as a one-letter/icon tag: **D** (dictionary), **R** (regex), **M** (model-inferred) — with a hover tooltip spelling out the full source and, for dictionary hits, the matched term (surfacing AI Pipeline §8's "matched dictionary entry" explainability point directly in the UI, not buried in a details drawer).
- The currently-playing word (driven by the video/audio `<currentTime>`) gets a distinct "playhead" underline independent of the confidence color, so playback position and detection status are visually decoupled and never compete for the same color channel.

9.3 Detection Detail Panel

Clicking a highlighted span (or a row in an optional flat list view, toggle available for users who prefer scanning a list over inline transcript reading) opens the detail panel showing:

- Detected text, exact timestamp range (editable — fine-tune via small +/- stepper buttons in 50ms increments, satisfying FR-TLE-06), language tag.
- **Confidence score shown as both the color-coded badge and the literal number** (e.g., "0.94 — High confidence, dictionary match") — showing the raw number alongside the color is deliberate: power users (the Priya/Professional Editor persona from PRD §3) want the precise value, casual users get the at-a-glance color.
- Replacement mode dropdown scoped to *this detection only* (FR-TLE-05), defaulting to the file-level setting from Section 8.2 but independently overridable.
- **Dismiss** (soft "not profane" — sets `is_dismissed: true` via `PATCH /detections/{id}`) and **Delete** are presented as the same single action to the user (the API distinction between dismiss-and-keep-audit-trail vs. hard delete, API Design §5.5, is an implementation detail the UI does not need to expose as two separate buttons).

9.4 Adding a Missed Detection (FR-TLE-04)

User selects a text range directly in the transcript (native text selection, not a custom widget — minimizes learning curve) or drags a range marker directly on the waveform; either path opens a small inline form (detected text pre-filled from the selection, start/end pre-filled from the selection's word timestamps, replacement mode defaulting to the file-level setting) and calls `POST /files/{id}/detections`. The resulting span renders with a distinct visual treatment (solid `--brand-primary` outline, not the confidence color scale, since a user-added detection has no AI-generated confidence score to visualize — this is intentional: manually added detections are never confused with AI-inferred ones).

9.5 Confidence Filtering (FR-PDE-06)

A slider in the sticky toolbar ("Show detections above: [0.5] confidence") re-filters which highlights render, without re-fetching data — all detections are already loaded client-side from the single `GET /files/{id}/detections` call, and the filter is a pure client-side render concern. This lets a cautious user (Priya, bulk/precision-focused) start at a high threshold and progressively lower it to review borderline cases in a deliberate second pass, rather than being shown everything at once.

9.6 Autosave, Undo/Redo, Regenerate

- Every edit action pushes onto a local undo/redo stack (`hooks/useTimelineEditor.js`, `store/editorStore.js`) supporting at least the last 10 actions (SRS-FR-TLE-09).
- Every 30 seconds (and on tab blur), the current in-progress edit state (not yet "confirmed" — see below) is serialized and sent to `POST /files/{id}/editor/autosave` (FR-TLE-08); on load, `GET /files/{id}/editor/autosave` is checked first and, if a draft newer than the last confirmed save exists, the user is prompted: "Restore your unsaved edits from [time]?"
- The bottom toolbar's **Regenerate** button batches all pending edits into a single `PATCH /files/{id}/detections/batch` call (API Design §5.4), then calls `POST /files/{id}/regenerate` (§4.6), and transitions the screen into a compact inline "Regenerating..." state (reusing the same WebSocket-driven stage indicator pattern from Section 8.3, but scoped to Stage 8–9 only per AI Pipeline §12/§14 — the UI's copy reflects this, showing "Applying edits" and "Finalizing export," never re-showing "Transcribing speech," which correctly communicates to the user that their edits are being applied quickly rather than the whole file being reprocessed).

9.7 Inline Preview

Once regeneration completes, an inline `<video>/<audio>` player swaps in showing the *censored* output directly in the same screen (no forced navigation), satisfying PRD §4.7 AC-1 ("edits are reflected in a live preview before final export").

10. History Page (`((app)/history/page.js)`)

Table/list view over `(GET /files)` (API Design §4.3) with search input, status filter chips, and sort control — directly implementing FR-DASH-02. Each row: filename, upload date, status pill, detected language tag, quick actions (Open Editor / Download / Delete). Pagination footer reflects the `(meta.total_pages)` envelope (API Design §2.5).

11. Settings & Profile (`((app)/settings/page.js)`, `((app)/profile/page.js)`)

- **Profile:** display name, email (read-only if OAuth-provisioned), account deletion action (a clearly separated, red-accented "Danger Zone" card — satisfies SRS-NFR-LEGAL-02's right-to-delete requirement with appropriately weighted visual friction, i.e., a confirmation modal requiring the user to type the word "delete").
 - **Settings:** Preset management (list/create/edit/delete over `(/presets)` endpoints, §9 of API Design), theme toggle (light/dark/system), and default replacement-mode preference (a pure client-side convenience default applied when the Upload page's Configuration Panel first renders — not a new backend field).
-

12. Admin Panel (`((admin)/admin/*)`)

Consistent shell (Section 4.1) across five sub-screens, all gated on `(role: admin)`:

12.1 User Management (`((admin)/users/page.js)`)

Paginated table (`(GET /admin/users)`) with search + status filter, row action to disable/enable a user (`(PATCH /admin/users/{id})`) behind a confirmation modal — mutating another user's account is always a two-step action, never a single click.

12.2 Profanity Dictionary (`((admin)/dictionary/page.js)`)

This screen is the direct admin-facing counterpart to AI Pipeline §8's "admin-editable, no-deployment-required" design goal. Table view (`(GET /admin/dictionary)`) filterable by language/active-status, inline "Add Term" form (term, language, severity — `(POST /admin/dictionary)`) and inline toggle for `(is_active)` (`(PATCH)`). A visible note near the top of this screen states plainly: *"Changes here take effect within 5 minutes for new uploads"* — setting accurate expectations given the in-memory cache TTL defined in AI Pipeline §8, rather than implying instant global effect.

12.3 Files Overview (`((admin)/files/page.js)`)

Cross-user paginated file table (`(GET /admin/files)`), filterable by status/user — a monitoring view, not an editing one; no destructive actions live here beyond viewing.

12.4 Analytics (`((admin)/analytics/page.js)`)

The one screen in the product that *does* use charts (justifying the earlier decision to keep the Dashboard chart-free): a bar chart of `(uploads_per_day)`, a single stat card for

`avg_processing_time_seconds`), and a stacked/grouped chart of `detection_stats.by_language` — all sourced from `GET /admin/analytics` (API Design §10.8), giving the admin (and, for demo purposes, the evaluator persona) a visual read on system health and the multilingual detection mix the whole product is built around.

12.5 Feedback (`admin/feedback/page.js`)

Simple list/detail view over `GET`/`PATCH /admin/feedback`, status chip (`open`/`reviewed`/`resolved`) changeable inline.

13. Component Inventory Cross-Reference

Maps this document's screens directly onto the folders already scaffolded in Folder Structure §3, so implementation has no ambiguity about where each piece lives:

Screen element	Component location
Dropzone / format validation UI	<code>components/upload/</code>
Recent files, credits/storage widgets	<code>components/dashboard/</code>
Transcript rendering, highlight spans, playback controls, edit toolbar	<code>components/timeline-editor/</code> (<code>TranscriptView.jsx</code> , <code>DetectionHighlight.jsx</code> , <code>PlaybackControls.jsx</code> , <code>EditActionsToolbar.jsx</code>)
Guest banner, navbar, sidebar, toasts	<code>components/shared/</code>
Admin tables/forms	<code>components/admin/</code>
Confidence color-scale logic, format constants	<code>lib/constants.js</code>
Editor undo/redo + draft state	<code>store/editorStore.js</code> , <code>hooks/useTimelineEditor.js</code>
WebSocket/job status subscription	<code>hooks/useJobStatus.js</code>
Form/response shape validation	<code>schemas/</code> (Zod — per Folder Structure v1.1 JS-only note)

14. Accessibility Notes (SRS-NFR-USE-02)

- Confidence is never communicated by color alone: every highlighted span also carries the literal score and source in its accessible name (`(aria-label="Detected: bakchodi, confidence 0.94, dictionary match")`), satisfying WCAG's color-independence guidance while still delivering the fast visual scan described in Section 9.2.
 - All interactive elements (dropzone, timeline spans, toolbar buttons) are keyboard-reachable and operable (Enter/Space to activate, arrow keys to step between detections in the transcript), per the WCAG 2.1 AA keyboard-navigation baseline.
 - Minimum contrast ratios verified for both light and dark token sets in Section 3.1, including the amber/rose confidence colors against their respective background tokens.
-

15. Next Document

Document 11: Development Roadmap — sequencing the implementation of the frontend screens specified here against the Release Plan (PRD §10) and the backend/AI modules from Documents 5–9, into a concrete sprint-by-sprint build order for the academic project timeline.

End of UI/UX Design Document